

JavaScript Object Constructors

[Back to JS page](#)

Table of Contents

- [JavaScript Object Constructors](#)
 - [Object Constructor Functions](#)
 - [Example 1](#)
 - [Property Default Values](#)
 - [Example 2](#)
 - [Adding a Property to an Object](#)
 - [Example 3](#)
 - [Constructor Function Methods](#)
 - [Example 4](#)
 - [Built-in JavaScript Constructors](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Object Constructor Functions

To create an object type, use an **object constructor function**. It is considered good practice to name constructor functions with an upper-case first letter:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Constructor Functions</h4>
<p id="demo"></p>

<script>
// Constructor function - named with uppercase first letter
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

// Create objects using the constructor
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Jane", "Doe", 48, "green");

document.getElementById("demo").innerHTML =
  "Father: " + myFather.firstName + " " + myFather.lastName + ", " + myFather.age + "<br>" +
  "Mother: " + myMother.firstName + " " + myMother.lastName + ", " + myMother.age;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Property Default Values

Constructor functions can have default property values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Property Default Values</h4>
<p id="demo"></p>

<script>
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
  this.nationality = "American"; // Default value
}

const myFather = new Person("John", "Doe", 50, "blue");
const mySister = new Person("Emily", "Doe", 25, "blue");

document.getElementById("demo").innerHTML =
  "Father: " + myFather.nationality + "<br>" +
  "Sister: " + mySister.nationality + "<br><br>" +
  "Both share the default nationality property";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding a Property to an Object

You can add a new property to an existing object. It only applies to that object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding a Property to an Object</h4>
<p id="demo"></p>

<script>
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

// Add a property to myFather only (not the constructor)
myFather.nationality = "American";

document.getElementById("demo").innerHTML =
  "Father nationality: " + myFather.nationality + "<br>" +
  "Mother nationality: " + myMother.nationality + " (undefined)<br><br>" +
  "Property added to myFather only";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Constructor Function Methods

You can define methods inside a constructor function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constructor Function Methods</h4>
<p id="demo"></p>

<script>
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.fullName = function() {
    return this.firstName + " " + this.lastName;
  };
  this.getAge = function() {
    return this.age;
  };
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

document.getElementById("demo").innerHTML =
  "Father: " + myFather.fullName() + " (" + myFather.getAge() + ")<br>" +
  "Mother: " + myMother.fullName() + " (" + myMother.getAge() + ")";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Built-in JavaScript Constructors

JavaScript has built-in constructors for native objects:

Constructor	Example
<code>Object()</code>	<code>new Object()</code>
<code>String()</code>	<code>new String("Hello")</code>
<code>Number()</code>	<code>new Number(42)</code>
<code>Boolean()</code>	<code>new Boolean(true)</code>
<code>Array()</code>	<code>new Array(1, 2, 3)</code>
<code>Date()</code>	<code>new Date()</code>

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Built-in JavaScript Constructors</h4>
<p id="demo"></p>

<script>
let text = "Built-in constructors:<br><br>";

const obj = new Object();
text += "new Object(): " + obj + "<br>";

const str = new String("Hello");
text += "new String('Hello'): " + str + "<br>";

const num = new Number(42);
text += "new Number(42): " + num + "<br>";

const bool = new Boolean(true);
text += "new Boolean(true): " + bool + "<br>";

const arr = new Array(1, 2, 3);
text += "new Array(1, 2, 3): " + arr + "<br>";

const date = new Date();
text += "new Date(): " + date;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Object Constructors](#)